Private Decentralized Inference on Consumer Hardware

Enabling Confidential Computation on Third-Party Apple Silicon via Software Access Path Elimination

Gajesh Naik

Eigen Labs

gajesh@eigenlabs.org

March 2026

Abstract

We present a system and method for private machine learning inference on a decentralized network of consumer hardware. The core problem: a user wants to run an AI model on someone else's Mac, but the machine's owner—who has root access and physical custody—should not be able to see the user's prompts or the model's responses. The machines lack hardware Trusted Execution Environments (TEEs) for application-level memory isolation.

Our approach eliminates every software path through which inference data could be observed. The inference engine runs directly inside a single hardened process—no subprocess, no local server, no inter-process communication. The macOS kernel blocks all external access to this process's memory: debuggers are denied at the system call level (`PT_DENY_ATTACH`), memory-reading APIs are blocked by Hardened Runtime, and these protections cannot be disabled without rebooting, which terminates the process and erases its data. We prove that System Integrity Protection, once verified, is immutable for the process lifetime (Theorem 1).

A four-layer attestation architecture verifies each machine in the network: (1) a Secure Enclave signature proves hardware-bound identity, (2) Apple's MDM framework independently confirms the machine's security configuration, (3) ACME `device-attest-01` produces an Apple-signed certificate chain proving the device is genuine Apple hardware with security features intact, and (4) periodic challenge-response confirms nothing has changed since enrollment.

After eliminating all software access paths, the only remaining attack requires physically probing the memory chips—which are soldered directly into Apple's System-on-Chip package. This is the same residual threat model accepted by Apple's Private Cloud Compute [1] for Siri and Apple Intelligence. We validate the system end-to-end on production hardware with live inference, demonstrating negligible security overhead (12 ms per request).

1 Introduction

When a user sends a prompt to a cloud AI service, that prompt travels to a server owned and operated by someone else. The user's privacy rests on a contractual promise—a privacy policy—not a technical guarantee. Hardware Trusted Execution Environments (TEEs) such as Intel TDX [6], AMD SEV-SNP [7], and NVIDIA Confidential Computing [8] can provide cryptographic memory isolation, but these technologies exist only on enterprise server hardware costing $14,000 or more.

Meanwhile, over 100 million Apple Silicon Macs have been sold since 2020. With 64–256 GB of unified memory and 273–819 GB/s memory bandwidth, they can run models with up to 235 billion parameters at interactive speeds. These machines sit idle most of the day. A decentralized network that lets Mac owners contribute their idle compute for AI inference—while guaranteeing that they cannot see the prompts being processed—would make private inference available at a fraction of the cost of cloud TEEs.

The challenge is that Apple Silicon provides no hardware TEE accessible to third-party applications:

- The **Secure Enclave** provides hardware-bound P-256 key generation and ECDSA signing, but cannot encrypt main system memory or provide isolated execution for arbitrary code.
- **App Attest** (`DCAppAttestService`) returns `false` for `isSupported` on macOS—it is available only on iOS and iPadOS.
- No public API exposes boot measurements or PCR-style remote attestation.

We present a system that makes this possible. Rather than encrypting memory (which Apple Silicon cannot do for third-party code), we *eliminate every software mechanism* through which the machine's owner could observe inference data—the same design philosophy Apple employs in Private Cloud Compute [1]. Apple's PCC removes all software interfaces (shell, debugger, persistent storage)

that could access inference data. We apply this principle to a harder problem: in PCC, Apple owns the hardware and controls physical access; in our system, the hardware owner is the adversary.

Contributions. This paper makes the following contributions:

1. A **decentralized private inference architecture** that enables confidential AI computation on a network of third-party Apple Silicon machines, where each machine's owner is assumed to be adversarial.
2. A **formal security model** for this setting, including a proof that System Integrity Protection provides runtime immutability under standard assumptions (Theorem 1), and a complete enumeration of the software attack surface.
3. An **in-process inference design** that embeds the inference engine directly in a hardened process, eliminating all inter-process communication channels that could be observed.
4. A **four-layer attestation architecture** combining Secure Enclave signatures, MDM-based independent verification, ACME `device-attest-01` hardware attestation with Apple-signed certificate chains, and periodic challenge-response with fresh security state.
5. A **combined enrollment protocol** that unifies MDM enrollment and ACME device attestation into a single configuration profile, reducing trust establishment to a one-step process.
6. **End-to-end validation** on production hardware (Apple M2 and M4 Max) with live inference, demonstrating that the security mechanisms introduce negligible overhead.
7. A **cost analysis** showing that Mixture-of-Experts models create 6–32× cost advantages on unified-memory hardware versus cloud GPUs, establishing economic feasibility.

2 Related Work

Hardware TEE-based inference. Intel TDX [6] and AMD SEV-SNP [7] encrypt VM memory and provide remote attestation. NVIDIA Confidential Computing [8] extends this to GPU memory. These approaches provide strong isolation but require enterprise server hardware. Our work achieves comparable privacy guarantees on consumer hardware without memory encryption.

Decentralized compute networks. Akash [2] provides general compute with TEE support in development. io.net [3] aggregates GPU resources without hardware security guarantees. Ritual [4] uses zero-knowledge proofs and Intel SGX. Bittensor [5] employs stake-weighted consensus for result verification. None address inference privacy on consumer hardware lacking TEEs.

Apple Private Cloud Compute. PCC [1] achieves inference privacy on Apple Silicon servers (M2 Ultra) without memory encryption through five requirements: stateless computation, enforceable guarantees, no privileged runtime access, non-targetability, and verifiable transparency. Implementation includes: no persistent storage, no SSH or shell access, immutable OS via Signed System Volume, cryptographic erasure on reboot, and OHTTP anonymization. Our work differs in that PCC operates in *Apple-controlled data centers* with physical security (access controls, tamper detection), while our system must defend against the hardware owner as adversary.

Cryptographic approaches. Fully Homomorphic Encryption [9] introduces 10^4–$10^6\times$ computational overhead. Secure Multi-Party Computation [10] requires multiple non-colluding servers. zkLLM [11] and DeepProve-1 [12] verify inference integrity via zero-knowledge proofs but do not protect input privacy during computation. These remain impractical for interactive LLM inference at current performance levels.

3 Threat Model and Assumptions

The fundamental challenge in decentralized inference is that the person running the computation is not the person who owns the data. A provider contributes their Mac's compute, but should learn nothing about the prompts processed on it. This section formalizes what we assume about each party and what a malicious provider can attempt.

3.1 Actors

Definition 1 (System Actors)**.** *We define four classes of actor:*

- $\mathcal{C}$ *(Consumer): The party submitting inference requests. Trusted—they have no incentive to attack their own data.*
- $\mathcal{K}$ *(Coordinator): The routing and verification service. Trusted via hardware: runs on Intel TDX [6] with cryptographic attestation of the deployed container image. The coordinator's memory is encrypted by the TEE hardware; neither the cloud operator nor the system operator can access it.*
- $\mathcal{P}$ *(Provider): The Mac owner contributing compute.* ***Assumed adversarial****—they own the hardware, have root access, and may actively attempt to read inference data. This is the strongest practical assumption: we guarantee privacy even if the provider is trying to break it.*
- $\mathcal{A}$ *(Apple): Trusted for the correctness of Secure Enclave hardware, SIP implementation, Kernel Integrity Protection, Hardened Runtime enforcement, IOMMU policy, code signature verification, and the Managed Device Attestation certificate authority.*

3.2 Adversary Capabilities

Definition 2 (Threat Model)**.** *The adversarial provider $\mathcal{P}$ can:*

- *Execute arbitrary code as root on the Mac*
- *Install any user-level software (subject to SIP restrictions)*
- *Reboot the machine at any time*
- *Physically access the machine exterior and ports*
- *Attempt to intercept or modify network traffic*

The adversary $\mathcal{P}$ cannot*:*

- *Exploit unpatched kernel vulnerabilities (Assumption 3)*
- *Modify Secure Enclave firmware or extract its keys*
- *Desolder LPDDR5x memory from the SoC package without destroying it*
- *Compromise Apple's Managed Device Attestation signing infrastructure*

Definition 3 (Assumptions)**.** ***Assumption 1*** *(Kernel Integrity): The macOS kernel has no unpatched vulnerabilities that allow SIP bypass, Hardened Runtime bypass, or KIP circumvention. Apple has patched all known SIP bypass CVEs (e.g., CVE-2022-22583, CVE-2022-42821, CVE-2023-32369) within weeks of disclosure.*

Assumption 2 *(Secure Enclave Integrity): The Secure Enclave hardware and firmware correctly implement key generation, signing, and key isolation. This is analogous to trusting Intel SGX or ARM TrustZone hardware.*

Assumption 3 *(Apple Attestation CA Integrity): Apple's Enterprise Attestation Root CA private key is not compromised, and Apple's attestation servers correctly verify device properties before issuing certificates. This is analogous to trusting any certificate authority in the Web PKI.*

3.3 Security Goal

In plain terms: a provider processes a prompt on their Mac, returns the result, and learns nothing about what the prompt said or what the model replied. More precisely:

Definition 4 (Privacy Property)**.** *Let x be an inference input (prompt) and $y = f(x)$ the model output. The system satisfies the* privacy property *if, for all polynomial-time adversaries $\mathcal{P}$:*

$$\Pr[\mathcal{P} \text{ learns } x \text{ or } y] \leq negl(\lambda) \tag{1}$$

where λ is the security parameter, under Assumptions 1–3, excluding physical memory probing attacks requiring destructive hardware modification.

4 System Architecture

The decentralized network comprises three principal components: a *coordinator* that routes consumer requests to providers and verifies their attestations, *provider agents* running on Apple Silicon Macs that execute inference, and a *Secure Enclave module* that gives each provider a hardware-bound cryptographic identity. Figure 1 depicts the complete architecture.

4.1 In-Process Inference

Traditional inference serving architectures (vLLM, Ollama, llama.cpp) run the engine as a separate HTTP server process. This creates two attack surfaces: (a) localhost TCP traffic can be captured via `tcpdump`, which functions even with SIP enabled; and (b) the subprocess binary can be replaced with a version that logs requests.

Our architecture eliminates both attack surfaces by embedding the inference engine directly in the provider process. The Python MLX inference engine is loaded via PyO3 [13] foreign function interface bindings within the Rust provider binary:

```
# Executed inside hardened Rust process via PyO3
import mlx_lm, builtins

# Model loaded in-process
builtins._model, builtins._tok = \
    mlx_lm.load("model-path")

# Inference in protected memory
result = mlx_lm.generate(
    builtins._model, builtins._tok,
    prompt=prompt, max_tokens=n)
```

Listing 1: Inference executes inside the hardened process address space. No subprocess, HTTP server, socket, or pipe is created.

The Python interpreter, model weights (up to 76 GB for a 122B-parameter model at 4-bit quantization), tokenizer, and all intermediate activations reside in a single process address space. This space is protected by three kernel-enforced mechanisms described in Section 4.2.

4.1.1 Python Path Isolation

A provider could install a modified inference library that logs all inputs. Since such code would execute *inside* the hardened process, the process-level protections would shield the attacker's logging code from external inspection.

We prevent this by restricting `sys.path` at interpreter initialization to include only packages from the signed application bundle, excluding the system `site-packages` directory. Any modification to the application bundle invalidates its code signature; SIP prevents macOS from executing binaries with invalid signatures (Section 5.1).

4.1.2 Memory Sanitization

After each inference request completes, buffers containing prompts and model outputs are zeroed using `write_volatile` operations (preventing compiler dead-store elimination) followed by a sequential-consistency

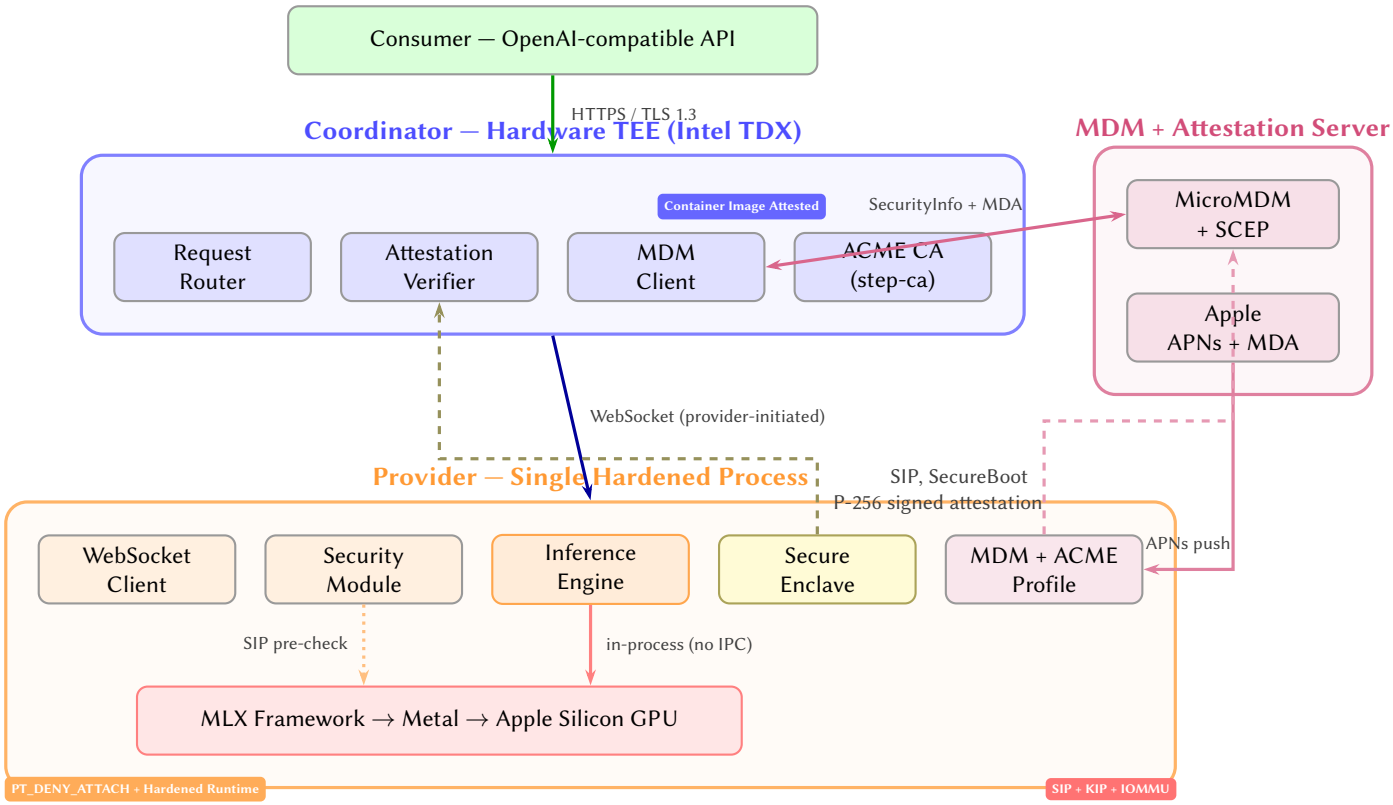

Figure 1: System architecture. The consumer connects via HTTPS to a coordinator running in a hardware TEE. The coordinator routes requests to provider Macs over provider-initiated WebSocket connections. Each provider runs inference in a single hardened process protected by PT_DENY_ATTACH, Hardened Runtime, SIP, KIP, and IOMMU. The Secure Enclave provides hardware-bound P-256 identity. MDM and ACME attestation independently verify security posture through Apple's infrastructure.

memory fence to ensure the writes are committed before returning.

4.2 Access Path Elimination

Three kernel-enforced mechanisms jointly eliminate all software paths to the inference process's memory:

1. **`PT_DENY_ATTACH`** (ptrace constant 31): Invoked at process startup before any sensitive data is loaded. Instructs the macOS kernel to permanently deny all `ptrace` requests against this process, including from root. This blocks `lldb`, `dtrace`, and Instruments.

2. **Hardened Runtime**: The binary is code-signed with hardened runtime options and explicitly *without* the `com.apple.security.get-task-allow` entitlement. The kernel denies `task_for_pid()` and `mach_vm_read()` from any external process.

3. **System Integrity Protection (SIP)**: Enforces both of the above at the kernel level. With SIP enabled, root cannot circumvent Hardened Runtime protections, load unsigned kernel extensions, or modify protected system binaries. Section 5.1 proves that SIP, once verified, is immutable for the process lifetime.

4.3 Coordinator Trust Boundary

The coordinator runs on a platform providing Intel TDX hardware TEEs on cloud infrastructure. The container image is cryptographically attested—consumers can verify that the coordinator executes unmodified code. The coordinator's memory is encrypted by the TDX hardware; neither the cloud infrastructure operator nor the system operator can access it.

The coordinator performs request routing, attestation verification, and key management. It receives encrypted inference requests from consumers and forwards them to verified providers. At no point does the coordinator need to decrypt inference payloads when end-to-end encryption is enabled (Section 8).

5 Formal Security Analysis

5.1 SIP Runtime Immutability

The security of our architecture depends on the property that SIP, once verified at process startup, cannot be disabled during the process lifetime.

Lemma 1 (SIP State Persistence). *SIP state $S \in \{\textsc{enabled}, \textsc{disabled}\}$ is stored in NVRAM and is immutable to all userspace and kernel code in the normal macOS boot*

environment.

Proof. The SIP state variable resides in the machine's NVRAM and is read by the Boot ROM during the secure boot chain. The only utility that modifies this variable is `csrutil`, which Apple restricts to execution within the Recovery Mode boot environment $\mathcal{R}$. In the normal macOS environment, `csrutil disable` returns an error. No other userspace or kernel API exposes write access to the SIP NVRAM variable when SIP is enabled, as SIP protects the NVRAM variables that control it—a self-reinforcing property. □

Theorem 1 (SIP Runtime Immutability)**.** *Under Assumption 1 (no unpatched kernel vulnerabilities), if SIP is verified as enabled at process startup time t_0, then SIP remains enabled at all times $t \in [t_0, t_{end}]$ during the lifetime of that process.*

Proof. By Lemma 1, SIP state is immutable in the normal boot environment. The only mechanism to change SIP state requires:

1. Reboot into Recovery Mode environment $\mathcal{R}$
2. Execute `csrutil disable` in $\mathcal{R}$
3. Reboot back to the normal environment

Step 1 requires a hardware reboot. Let $\mathcal{E}(t)$ denote the set of all running processes at time t. A reboot at time t_r terminates every process:

$$\forall\, p \in \mathcal{E}(t_r^-) : \neg\,\text{alive}(p,\, t_r^+) \tag{2}$$

The provider process $p^* \in \mathcal{E}(t_0)$. For p^* to observe $S = \textsc{disabled}$, a reboot must occur. But a reboot terminates p^*. Therefore:

$$S(t_0) = \textsc{enabled} \implies \forall\, t \in [t_0,\, t_{\text{end}}] :\; S(t) = \textsc{enabled} \tag{3}$$

□

Corollary 1 (Single Verification Sufficiency)**.** *A single SIP verification at process startup provides a security guarantee for the entire process lifetime. Per-request SIP checks serve as defense-in-depth but do not detect state changes, because none can occur under the theorem's conditions.*

5.2 Complete Attack Surface Analysis

Table 1 enumerates all known software-based attacks against the provider process and their corresponding defenses.

5.3 Self-Reinforcing Security Properties

Several verification mechanisms exhibit *circular dependency* that strengthens rather than weakens the security model:

Proposition 1 (MDM Verification Circularity)**.** *Spoofing the MDM `SecurityInfo` response to report SIP as enabled when it is disabled requires modifying system frameworks in `/System/Library/`, which requires SIP to be disabled. Therefore, a provider cannot simultaneously (a) have SIP disabled and (b) cause the MDM framework to report SIP as enabled.*

This circularity ensures that the MDM verification is unforgeable precisely in the cases where it matters most. Combined with Authenticated Root Volume verification (Section 6.2.2), which proves the entire system volume matches Apple's sealed build, this creates a defense-in-depth where each layer independently validates the others.

5.4 Provider Verification Requirements

Definition 5 (Verified Provider)**.** *A provider achieves* verified *status if and only if all of the following conditions hold:*

1. *Secure Enclave P-256 attestation signature is valid (Section 6.1)*
2. *Binary hash matches a known blessed version*
3. *Device is enrolled via the combined enrollment profile (Section 7)*
4. *MDM SecurityInfo confirms: SIP enabled, Secure Boot at "full" level, Authenticated Root Volume enabled*
5. *Apple MDA certificate chain is valid and serial number matches self-reported attestation (Section 6.3)*
6. *MDM-reported security posture matches self-reported attestation with no discrepancies*
7. *Periodic challenge-response succeeds with fresh SIP and Secure Boot status (Section 6.4)*

Providers that fail any verification receive no inference requests. No degraded or partial-trust mode is supported.

6 Four-Layer Attestation Architecture

The attestation architecture provides four independent layers of verification, each addressing distinct threat vectors. Table 2 summarizes the layers and their properties.

6.1 Layer 1: Secure Enclave Attestation

On first execution, the provider generates a P-256 ECDSA signing key inside Apple's Secure Enclave via CryptoKit. The private key never leaves the Secure Enclave hardware—the `dataRepresentation` stored on disk is an opaque handle that functions only on the device that created it. The provider constructs a signed attestation blob:

Three fields merit particular attention:

serialNumber. The hardware serial number enables the coordinator to correlate the device with its MDM enrollment record and request independent security verification.

Table 1: Complete software attack surface on Apple Silicon. All listed attacks are blocked under Assumptions 1–2. The final row represents the residual physical attack.

Attack Vector	Defense Mechanism	Enforced By
Attach debugger (lldb, dtrace, Instruments)	`ptrace(PT_DENY_ATTACH)` at process startup	Kernel
Read memory via Mach APIs (`task_for_pid`, `mach_vm_read`)	Hardened Runtime without `get-task-allow` entitlement	Kernel
Intercept IPC between processes	No IPC exists—inference is in-process	Architecture
Modify provider binary to add logging	Code signing + SIP: macOS refuses modified signed binaries	Kernel + SIP
Replace binary with malicious version	Binary SHA-256 in Secure Enclave-signed attestation; coordinator rejects	Coordinator
Inject malicious Python package	`sys.path` locked to signed app bundle; system site-packages excluded	Process + SIP
Load unsigned kernel extension	SIP blocks all unsigned kexts on Apple Silicon	SIP
Modify kernel code at runtime	Kernel Integrity Protection: hardware denies writes to kernel pages after boot	Hardware
Disable SIP	Requires reboot into Recovery Mode, terminating all processes (Theorem 1)	Hardware
Read physical memory via `/dev/mem`	Device node does not exist on Apple Silicon	Hardware
DMA-based extraction via Thunderbolt/PCIe	Per-device IOMMU with default-deny; unauthorized DMA triggers kernel panic	Hardware
Physical memory probing	*LPDDR5x soldered into SoC package; desoldering is destructive*	*Physical*

Table 2: Four-layer attestation architecture. Each layer addresses threats not covered by the others.

Layer	Mechanism	Threat Addressed
1	SE P-256 ECDSA	Hardware-bound identity; replay prevention
2	MDM SecurityInfo	Independent SIP/SecureBoot verification
3	ACME device-attest-01	Apple-signed hardware provenance
4	Challenge-response	Liveness; continuous security state

This field bridges Layer 1 (self-reported) to Layers 2 and 3 (independently verified).

binaryHash. The SHA-256 hash of the running provider binary. The coordinator maintains a set of blessed binary hashes for each release. A non-matching hash indicates modified code, and the provider is rejected. This prevents an adversary from patching the binary to bypass security checks (e.g., removing SIP verification or Python path isolation).

encryptionPublicKey. Binds the provider's X25519 encryption key (used for end-to-end encrypted inference, Section 8) to the same Secure Enclave identity. The coordinator verifies this matches the key in the registration message, proving the same physical device controls both the signing and encryption keys.

Table 3: Attestation blob fields. JSON-encoded with deterministic key ordering (alphabetical) and signed by the Secure Enclave P-256 key.

Field	Type	Purpose
`authenticatedRootEnabled`	bool	Sealed system volume (ARV)
`binaryHash`	hex string	SHA-256 of running binary
`chipName`	string	Hardware identity
`encryptionPublicKey`	base64	X25519 key bound to this identity
`hardwareModel`	string	Machine model identifier
`osVersion`	string	macOS version
`publicKey`	base64	P-256 public key (raw X‖Y)
`secureBootEnabled`	bool	Secure Boot status
`secureEnclaveAvailable`	bool	SE hardware present
`serialNumber`	string	Device serial for MDM cross-ref
`sipEnabled`	bool	SIP status
`systemVolumeHash`	hex string	APFS snapshot hash
`timestamp`	ISO 8601	Freshness

6.1.1 *Cross-Language Signature Verification*

The attestation protocol spans three programming languages: Swift (Secure Enclave signing), Rust (provider agent), and Go (coordinator verification). All three must produce byte-identical JSON representations for signature verification to succeed. We enforce this through:

- Struct fields declared in alphabetical order (by JSON key name) in all three languages
- Swift's `JSONEncoder` configured with `.sortedKeys` output formatting

- Go coordinator preserves raw attestation bytes (`json.RawMessage`) for verification, with a `marshalSortedJSON` fallback using `map[string]interface{}` (Go 1.12+ marshals map keys in sorted order)
- `omitempty` handling: nil fields omitted (not set to null)
- ISO 8601 timestamp format with consistent precision
- Base64 standard alphabet (no URL-safe variant); raw P-256 key format (64 bytes: X‖Y, no 0x04 prefix)

6.1.2 Coordinator Verification Procedure

The coordinator verifies each attestation through the following steps:

1. Decode the P-256 public key from the attestation blob (64-byte raw X‖Y format)
2. Hash the original JSON blob bytes with SHA-256
3. Verify the DER-encoded ECDSA signature against the hash
4. Enforce: `secureEnclaveAvailable` = true, `sipEnabled` = true, `secureBootEnabled` = true
5. Verify: `encryptionPublicKey` matches the registration message
6. Verify: `binaryHash` is in the set of blessed versions
7. Verify: `timestamp` is within acceptable freshness window
8. If all pass, proceed to Layer 2 verification

6.2 Layer 2: MDM-Based Independent Verification

Secure Enclave attestation (Layer 1) has two inherent limitations: (a) the coordinator cannot prove the P-256 signing key resides in the Secure Enclave rather than in software, since both produce identical ECDSA signatures; and (b) the SIP status in the attestation blob is obtained via `csrutil status`, a software check that could theoretically be spoofed on a compromised system.

Layer 2 addresses both limitations through *independent verification*: the coordinator queries the device's security posture through Apple's MDM framework, which returns data from the OS MDM subsystem—not from the provider software.

6.2.1 SecurityInfo Command

Apple's MDM `SecurityInfo` command [14] returns the following fields, as reported by the operating system's MDM client:

The coordinator cross-checks these values against the self-reported attestation blob. Any discrepancy results in immediate rejection.

6.2.2 Authenticated Root Volume

The Authenticated Root Volume (ARV) seals the macOS system volume with a Merkle tree of SHA-256 hashes. Any modification to any system file—including `mdmclient`, `csrutil`, and kernel caches—breaks the seal and prevents the volume from mounting. ARV provides a stronger guarantee than SIP alone: SIP prevents *runtime* modifications, while ARV proves the *entire system volume* is Apple's unmodified build.

Table 4: Fields returned by the MDM `SecurityInfo` command. The coordinator requires SIP enabled, Secure Boot "full," and Authenticated Root Volume enabled.

Field	Verification Purpose
`SIPEnabled`	SIP active—root cannot modify `/System` or bypass Hardened Runtime
`SecureBootLevel`	"full": only Apple-signed code at boot
`AuthRootVolume`	Signed System Volume Merkle tree intact
`FDE_Enabled`	FileVault disk encryption active
`RecoveryLock`	Recovery Mode requires password
`FirewallEnabled`	Application firewall active

The `systemVolumeHash` field in the attestation blob contains the cryptographic hash of the APFS system volume snapshot, uniquely identifying the exact macOS build. The coordinator can cross-reference this against known-good builds published by Apple.

6.3 Layer 3: ACME Device Attestation (device-attest-01)

Layers 1 and 2 verify security *posture* (is SIP enabled?) but do not provide cryptographic proof of hardware *identity* (is this a genuine Apple device?). Layer 3 addresses this through Apple's Managed Device Attestation (MDA) protocol, implemented via the ACME `device-attest-01` challenge type [16].

6.3.1 Protocol Overview

The ACME `device-attest-01` challenge extends RFC 8555 (Automatic Certificate Management Environment) with hardware attestation. When the device receives an ACME challenge:

1. The device generates a P-384 ECDSA key pair inside the Secure Enclave (`HardwareBound = true`, `Attest = true` in the ACME profile payload)
2. The device contacts Apple's attestation servers
3. Apple verifies the device's identity and security properties using the Secure Enclave's hardware attestation capability
4. Apple issues a DER-encoded X.509 certificate chain:
   - **Leaf**: Device certificate with Apple-assigned OIDs encoding hardware properties

- **Intermediate**: Apple Enterprise Attestation Sub CA 1 (P-384)
- **Root**: Apple Enterprise Attestation Root CA (P-384, valid until 2047)

5. The ACME CA (step-ca [18], co-located with the coordinator) validates the Apple-signed attestation and issues a final certificate binding the hardware-bound key to the device identity

6.3.2 Certificate OID Fields

The leaf certificate in Apple's attestation chain encodes device properties as X.509 extensions with Apple-assigned OIDs:

Table 5: Apple-assigned OIDs in MDA leaf certificates. These values are signed by Apple's Enterprise Attestation Root CA and cannot be forged.

OID Suffix	Property	Verification Use
100.8.9.1	Serial Number	Device identity
100.8.9.2	UDID	Device identity
100.8.10.1	OS Version	OS currency
100.8.10.2	SepOS Version	SE firmware version
100.8.10.3	LLB Version	Boot chain integrity
100.8.11.1	Freshness Code	Replay prevention
100.8.13.1	SIP Status	SIP independently verified
100.8.13.2	Secure Boot	Boot security level
100.8.13.3	Kext Status	Kernel extension policy

All OIDs prefixed with 1.2.840.113635.

6.3.3 Verification Properties

MDA provides the strongest available verification: Apple's own servers cryptographically attest the device's identity and security properties. A compromised operating system cannot forge this certificate chain—the private key for Apple's root CA exists only within Apple's infrastructure (Assumption 3).

The coordinator verifies the certificate chain against the embedded Apple Enterprise Attestation Root CA, extracts the OID-encoded properties, and cross-checks the serial number against the provider's self-reported attestation. The complete certificate chain is stored and exposed via a public API endpoint, enabling consumers to independently verify each provider against Apple's publicly available root CA certificate.

6.3.4 Comparison with SecurityInfo

Table 6 compares the two independent verification mechanisms.

Table 6: MDM SecurityInfo (Layer 2) vs. ACME MDA (Layer 3).

Property	SecurityInfo	MDA
Trust anchor	OS MDM client	Apple Root CA
Forgery requires	SIP disabled	Apple CA compromise
Proves identity	No	Yes (serial, UDID)
Proves security	Yes (SIP, boot)	Yes (SIP, boot, SepOS)
Latency	Seconds	Minutes (Apple servers)
Availability	Always	Requires enrollment

6.4 Layer 4: Continuous Challenge-Response

Layers 1–3 establish trust at enrollment time. Layer 4 provides continuous assurance that the provider's security posture has not changed since enrollment.

Every 5 minutes, the coordinator generates a random 32-byte nonce n and timestamp t, and sends an `attestation_challenge` message to the provider. The provider must:

1. Check current SIP status via `csrutil status`
2. Check current Secure Boot status
3. Compute signature $\sigma = \text{Sign}_{\text{SE}}(n \| t \| \text{pk})$ where pk is the registered public key
4. Return $(\sigma, n, \text{pk}, \text{sip_enabled}, \text{secure_boot_enabled})$ within 30 seconds

The coordinator verifies:

- Nonce n matches the sent value
- Public key pk matches the registered key
- ECDSA signature σ is valid
- `sip_enabled` = true (immediate untrust if false)
- `secure_boot_enabled` = true (immediate untrust if false)

Immediate untrust policy: If SIP or Secure Boot is reported as disabled in any challenge response, the provider is immediately marked untrusted with no grace period. Disabled SIP or Secure Boot indicates a deliberate reboot with weakened security (by Theorem 1, SIP cannot be disabled without rebooting).

Failure escalation: Three consecutive signature verification failures result in untrust marking. This catches providers experiencing connectivity issues versus providers that have been replaced or compromised.

Reconnection scenario: If a provider reboots (e.g., to disable SIP), disconnects, and reconnects, the reconnection triggers a fresh registration with a new attestation blob. If the new attestation reports SIP as disabled, the provider is rejected. If the provider lies about SIP status, the next MDM SecurityInfo query independently reveals the true state.

7 Combined Enrollment Protocol

A practical concern in deploying multi-layer attestation is user friction: requiring providers to install multiple configuration profiles creates opportunities for partial enrollment and increases abandonment. We address this with

a *combined enrollment protocol* that unifies all enrollment steps into a single Apple configuration profile.

7.1 Profile Structure

The coordinator generates a per-device `.mobileconfig` containing three payloads:

1. **SCEP payload** (`com.apple.security.scep`): Generates an RSA-2048 identity certificate via Simple Certificate Enrollment Protocol. This certificate authenticates the device to the MDM server.
2. **MDM payload** (`com.apple.mdm`): Enrolls the device with the MDM server using the SCEP identity certificate. The `AccessRights` bitmask is set to 1041 ($2^0 + 2^4 + 2^{10}$), granting only:
   - Bit 0: Inspect device (basic query)
   - Bit 4: Query device information (model, OS, serial)
   - Bit 10: Query security information (SecurityInfo command)

   The remaining 10 capability bits are explicitly unset. The MDM server **cannot** erase the device, lock it, change the passcode, install or remove profiles or applications, modify settings, or query network information.
3. **ACME payload** (`com.apple.security.acme`): Initiates the `device-attest-01` challenge. The payload specifies:
   - `KeyType = ECSECPrimeRandom`, `KeySize = 384`: P-384 key
   - `HardwareBound = true`: Key generated in Secure Enclave
   - `Attest = true`: Apple attestation requested
   - `Subject CN` = device serial number (for cross-referencing)
   - `DirectoryURL`: Points to the step-ca ACME server

7.2 Enrollment Flow

1. Provider runs the installation script, which detects the device serial number via `system_profiler`
2. Script requests a per-device profile from the coordinator: `POST /v1/enroll` with `{serial_number}`
3. Coordinator generates the combined profile with fresh UUIDs
4. macOS displays the profile contents and requested permissions to the user for review and consent
5. On installation, macOS processes all three payloads atomically:
   - SCEP certificate issued and stored in System Keychain
   - MDM enrollment completes; device checks in with MDM server
   - ACME challenge initiated; device contacts Apple for attestation
   - Apple-signed certificate chain returned and stored in Keychain
6. Provider is now eligible for all four layers of verification

7.3 Minimal Permission Principle

The MDM enrollment requests the minimum permissions necessary for security verification. Table 7 enumerates the complete MDM `AccessRights` bitmask.

Table 7: Apple MDM AccessRights bitmask. The system requests only bits 0, 4, and 10 (bold).

Bit	Permission	Ours	Typical
0	**Inspect device**	✓	✓
1	Install/remove configuration profiles	–	✓
2	Device lock and passcode removal	–	✓
3	Erase all data on device	–	✓
4	**Query device information**	✓	✓
5	Query network information	–	✓
6	Inspect provisioning profiles	–	✓
7	Install/remove provisioning profiles	–	✓
8	Inspect installed applications	–	✓
9	Restriction-related queries	–	✓
10	**Query security information**	✓	✓
11	Change device settings	–	✓
12	App management	–	✓

Providers can unenroll at any time via System Settings, which removes the profile and all associated certificates immediately and completely.

8 End-to-End Encryption

The system supports optional end-to-end encryption of inference payloads using NaCl Box (X25519 key agreement + XSalsa20-Poly1305 authenticated encryption) [17].

8.1 Protocol

For each inference request, the coordinator:

1. Generates an ephemeral X25519 key pair (sk_e, pk_e)
2. Generates a random 24-byte nonce n
3. Computes the shared secret and encrypts: $c = \text{Box}(m, n, pk_{\text{provider}}, sk_e)$
4. Sends $(pk_e, n \| c)$ to the provider

The provider decrypts using its persistent X25519 key (bound to its Secure Enclave identity via the attestation blob, Section 6.1).

8.2 Forward Secrecy

Each request uses a fresh ephemeral key pair. Compromise of any single ephemeral key does not enable decryption of past or future requests. The provider's long-term X25519 key is stored on disk with restricted permissions (0600);

its binding to the Secure Enclave identity is verified by the coordinator.

9 Provider Scoring and Routing

The coordinator selects providers using a composite score that incorporates trust verification, measured performance, and real-time system health.

9.1 Scoring Function

$$s = (1 - \ell) \cdot d \cdot \tau \cdot \rho \cdot w \cdot h \tag{4}$$

where:

- $\ell \in [0, 1]$: load factor (gradient based on concurrent request count)
- d: measured decode throughput (tokens/second) from registration benchmark
- τ: trust multiplier—1.0 for hardware-verified, 0.8 for self-signed, 0.5 for unverified
- $\rho \in [0, 1]$: composite reputation score (Section 9.3)
- w: warm model bonus—1.5 if the requested model is loaded in memory, 1.0 otherwise
- $h \in [0, 1]$: health factor derived from real-time system metrics

9.2 Health Factor

The health factor is computed from live system metrics reported in provider heartbeats (every 30 seconds):

$$h = (1 - m_p) \cdot (1 - c_u) \cdot t_m \tag{5}$$

where m_p is memory pressure (from `vm_stat`: ratio of active + wired + compressed pages to total), c_u is CPU utilization (1-minute load average divided by core count, clamped to $[0, 1]$), and t_m is a thermal multiplier derived from CPU speed limit (1.0 at nominal, linearly decreasing to 0.25 at critical thermal state).

9.3 Reputation Score

Provider reputation is a weighted composite of four factors:

$$\rho = 0.4 \cdot r_j + 0.3 \cdot r_u + 0.2 \cdot r_c + 0.1 \cdot r_t \tag{6}$$

where r_j is job success rate (successful/total, default 0.5 for new providers), r_u is uptime fraction ($\min(\text{uptime}/24\text{h}, 1.0)$), r_c is challenge-response success rate (Section 6.4), and r_t is response time factor (1.0 if $\leq$1s average, linearly degrading to 0.0 at 10s).

10 Comparison with Apple Private Cloud Compute

Table 8 compares the system's security properties with Apple PCC.

Table 8: Comparison with Apple Private Cloud Compute [1].

Property	Apple PCC	This Work
Hardware owner	Trusted (Apple)	Adversarial
Physical security	Facility controls	Soldered LPDDR5x
Coordinator TEE	N/A (Apple infra)	Intel TDX
OS immutability	SSV	SIP + ARV + SSV hash
Shell/debug access	Removed entirely	Blocked at kernel level
Memory encryption	None	None
Provider verification	Apple-signed	SE + MDM + MDA
Hardware provenance	Apple supply chain	Apple MDA certificates
Residual attack	Physical probing	Physical probing

The key distinction is who owns the hardware. In PCC, Apple owns the servers and controls physical access. In our system, the hardware belongs to an independent provider who may be adversarial. The four-layer attestation architecture (Section 6) compensates for the absence of physical security by providing multiple independent verification paths, culminating in Apple-signed hardware provenance certificates. The result is that consumers get the same residual threat model—physical memory probing only—whether they use Apple's private cloud or our decentralized network.

11 Implementation

The system is implemented in approximately 14,000 lines of code across five languages:

Table 9: Implementation components and languages.

Component	Language	Lines
Provider agent	Rust	4,500
Coordinator	Go	4,000
Verification frontend	TypeScript (Next.js)	2,500
Consumer SDK	Python	1,200
Secure Enclave module	Swift	600
macOS application	Swift/SwiftUI	1,500
Total		**∼14,300**

Coordinator. Implemented in Go. Runs within a hardware TEE (Intel TDX) with cryptographic container attestation. Exposes an OpenAI-compatible HTTP API for consumers and WebSocket endpoints for provider registration and messaging. The ACME CA is provided by step-ca [18] with the `device-attest-01` challenge type configured.

The MDM server is MicroMDM [19] with SCEP certificate issuance.

Provider agent. Implemented in Rust. Manages the inference engine lifecycle, WebSocket connection to the coordinator with automatic reconnection and exponential backoff, Secure Enclave attestation via the enclave CLI helper, hardware detection, system metrics collection, and challenge-response signing. Supports both embedded inference (via PyO3) and subprocess proxy modes.

Secure Enclave module. Implemented in Swift using CryptoKit's `SecureEnclave.P256.Signing.PrivateKey`. Provides a CLI interface invoked by the provider at startup: `dginf-enclave attest [-encryption-key <b64>] [-binary-hash <hex>]`. Generates the signed attestation blob with hardware identity and security posture.

Provider installation. A single shell command downloads a self-contained bundle (∼107 MB) including the provider binary, Secure Enclave CLI, and a bundled Python environment with the inference runtime. No system Python or package manager is required. The script detects the device serial number, requests a combined enrollment profile from the coordinator, and opens the profile for user review and installation.

12 Evaluation

12.1 End-to-End Validation

We validated the complete system on two Apple Silicon machines simultaneously: an AWS `mac2-m2.metal` instance (Apple M2, 24 GB unified memory, macOS 14.8.4) and a local M4 Max (48 GB unified memory, macOS 26.3). The validation sequence:

1. Both providers enrolled via combined profiles (SCEP + MDM + ACME, AccessRights = 1041)
2. Both generated Secure Enclave attestation blobs with serial numbers
3. Coordinator verified SE P-256 signatures → Layer 1 (self-signed trust)
4. Coordinator queried MDM; SecurityInfo confirmed SIP = true, SecureBoot = full, AuthenticatedRootVolume = true for both → Layer 2
5. ACME `device-attest-01` challenges completed; Apple's attestation servers returned DER certificate chains signed by Apple Enterprise Attestation Root CA for both devices → Layer 3
6. Coordinator verified Apple certificate chains against embedded root CA; serial numbers cross-checked against attestation blobs
7. Both providers reached fully verified status (all four layers)
8. Consumer inference requests routed to both providers via health-aware scoring
9. M4 Max: 92 tok/s decode (Qwen3.5 9B, Q4); M2: 22 tok/s
10. 8 concurrent requests completed with 100% success rate across both providers; 6,517 tokens generated in 76.2 s
11. Consumer disconnect triggered cancel propagation within 1 s: coordinator → provider → backend
12. Full attestation proof (including Apple MDA certificate chains) exposed at public API endpoint for independent consumer verification

12.2 Security Verification

Table 10: Security and performance measurements on two providers.

Verification	Result
PT_DENY_ATTACH	lldb attach denied
SIP status (both devices)	Enabled
Secure Boot (both devices)	Full
Authenticated Root Volume	Enabled
MDM SecurityInfo cross-check	Match
Apple MDA certificate chain	Valid (both devices)
MDA serial cross-check	Match
Challenge-response (5-min interval)	All passed
Performance	**Value**
Decode throughput (9B, M4 Max)	92 tok/s
Decode throughput (9B, M2)	22 tok/s
Time to first token	1.4 s
SIP check overhead	12 ms
Reconnect after disconnect	<1 s
Concurrent requests per provider	4
8 concurrent, 0 failures	100%

Security overhead is negligible. The SIP check (12 ms per request) is the largest per-request cost, dominated by subprocess invocation of `csrutil status`. By Corollary 1, this check could be performed once at startup without loss of security guarantee; the per-request check serves as defense-in-depth.

13 Economic Feasibility Analysis

We analyze the economic properties of the system to establish that decentralized inference on consumer hardware is economically viable at the price points required for adoption.

13.1 Marginal Cost Model

For providers with hardware already purchased, the marginal cost per million output tokens is:

$$C_{\text{local}} = \frac{P_{\text{elec}}}{R_{\text{tok}} \times 3{,}600} \times 10^6 \tag{7}$$

where P_{elec} is electricity cost ($/hr) and R_{tok} is decode throughput (tok/s). At U.S. average residential $0.15/kWh, an Apple Silicon workstation draws ∼100 W under inference load ($0.015/hr) and 5 W at idle.

13.2 Throughput Measurements

Table 11: Decode throughput (tok/s, Q4 quantization) on Apple Silicon. † = measured on live system; others bandwidth-scaled at 65% efficiency.

Model	Architecture	M4 Max 546 GB/s	M3 Ultra 819 GB/s
Qwen3.5 9B	Dense	92†	94
Qwen3.5 27B	Dense	21	32
Qwen3.5 35B-A3B	MoE (3B active)	101	152
Llama 3.1 70B	Dense	8	13
Qwen3.5 122B-A10B	MoE (10B active)	25	35
MiniMax M2.5 230B	MoE (10B active)	—	40

"—" = does not fit in memory.

Observation: Mixture-of-Experts (MoE) models are disproportionately efficient on unified-memory hardware. Qwen3.5 35B-A3B (3B active parameters) achieves 101 tok/s on M4 Max—faster than the dense Qwen3.5 27B at 21 tok/s, despite a larger total parameter count. For MoE architectures, only active expert weights traverse the memory bus per token.

13.3 Cost Comparison

Cloud inference providers price based on total parameter count. Local inference cost depends only on *active* parameters per token. For MoE models, this creates 6–32× cost advantages:

Table 12: Cost comparison: cloud inference vs. local (electricity only).

Model	Cloud $/M	Local $/M	Ratio
Qwen3.5 35B-A3B	$1.30	$0.04	32×
Qwen3.5 122B-A10B	$2.08	$0.09	23×
Qwen3.5 27B	$1.56	$0.20	7.8×
MiniMax M2.5 230B	$0.95	$0.16	6×
Llama 3.1 70B	$0.30	$0.49	0.6×

Dense models exceeding 32B parameters favor cloud infrastructure: H100 clusters with 3.35 TB/s memory bandwidth achieve 6× higher throughput than M4 Max (546 GB/s) for bandwidth-bound decoding. The economic sweet spot for decentralized inference is MoE models with small active parameter counts on large unified-memory hardware.

14 Limitations

Kernel vulnerability assumption. Our security model (Theorem 1) assumes no unpatched kernel vulnerabilities (Assumption 1). A macOS kernel zero-day could bypass SIP, Hardened Runtime, or KIP, undermining the trust model. This is an inherent limitation of software-enforced security boundaries; only hardware TEEs provide immunity to kernel-level attacks. We rely on Apple's track record of rapid patching.

Timing side channels. Our architecture protects token *content*—the provider cannot observe what tokens are generated. However, token *timing* is observable. Network packet intervals reveal approximate prompt length (from prefill duration), response length (from output packet count), and relative generation difficulty (from inter-token delays). We do not implement constant-time inference. Mitigations include response buffering and random jitter. This limitation is shared with Apple PCC and all non-TEE inference systems.

Single-device throughput. The Python GIL serializes inference to one request at a time when using the PyO3 embedding. Direct Rust FFI bindings to MLX's C++ API would eliminate this bottleneck and the Python package injection surface simultaneously.

MDA availability. ACME `device-attest-01` requires devices to be enrolled with an organization that has Apple attestation authority. Expanding to arbitrary consumer devices requires integration with Apple Business Manager or equivalent enrollment pathway.

15 Future Work

Request anonymization. OHTTP (RFC 9458) relays and RSA blind signatures would prevent the coordinator from linking consumer identity to request content, achieving non-targetability—Apple PCC's fourth requirement.

Multi-device inference. Thunderbolt 5 RDMA (<50 μs latency, macOS 26.2+) enables model sharding across multiple provider machines, extending capacity to 400B+ parameter models.

Encrypted model execution. Model weights encrypted at rest, with decryption keys released only to providers whose full attestation chain verifies. The provider's hardened process decrypts weights in memory, performs inference, and wipes them. The same protections that guard inference inputs also guard model intellectual property.

Direct MLX FFI. Replacing PyO3 with native Rust bindings to MLX's C++ API eliminates the Python interpreter, the GIL serialization bottleneck, the Python path isolation requirement, and the package injection attack surface.

General private compute. The access path elimination technique is not specific to inference. Any computation that can be embedded in a hardened process on Apple Silicon—fine-tuning on private data, evaluation on proprietary datasets, confidential data pipelines—can leverage the same security guarantees. The four-layer attestation architecture generalizes to any workload where computation must remain confidential from the machine's owner, positioning the decentralized network as a general-purpose private compute layer on consumer hardware.

16 Conclusion

We have presented a decentralized inference system in which consumers get private AI computation on machines they do not own or control. The machine owners—who have root access and physical custody—cannot observe the prompts or responses processed on their hardware.

The approach works by systematically eliminating every software path to inference data: in-process execution removes inter-process communication, kernel-enforced isolation blocks memory inspection, and operating system integrity guarantees are provably immutable at runtime (Theorem 1). A four-layer attestation architecture—Secure Enclave signatures, MDM-based independent verification, ACME `device-attest-01` with Apple-signed certificate chains, and continuous challenge-response—provides defense-in-depth where each layer independently verifies properties that the others cannot.

The residual attack surface reduces to physical memory probing of LPDDR5x soldered into Apple's System-on-Chip package. This is the same threat model accepted by Apple's Private Cloud Compute for Siri and Apple Intelligence queries. Our system achieves equivalent privacy guarantees on a decentralized network of independently owned machines.

The system is validated end-to-end on production hardware with live inference, demonstrating negligible security overhead. The economic analysis establishes that Mixture-of-Experts models create 6–32× cost advantages on unified-memory hardware, making decentralized private inference economically viable. The access path elimination technique generalizes beyond inference to any computation requiring confidentiality on third-party consumer hardware.

References

[1] Apple Security Engineering and Architecture. Private Cloud Compute: A new frontier for AI privacy in the cloud. *Apple Security Research*, 2024.

[2] Akash Network. The Decentralized Cloud. https://akash.network/, 2024.

[3] io.net. The Internet of GPUs. https://io.net/, 2024.

[4] Ritual. Infernet: Decentralized AI Infrastructure. https://ritual.net/, 2024.

[5] Y. Rao et al. Bittensor: A peer-to-peer intelligence market. *arXiv*, 2023.

[6] Intel Corporation. Intel Trust Domain Extensions (TDX). *Intel Architecture Specification*, 2023.

[7] D. Kaplan et al. AMD SEV-SNP: Strengthening VM isolation with integrity protection and more. *AMD White Paper*, 2020.

[8] NVIDIA. Confidential Computing on H100 GPUs. *NVIDIA Technical Blog*, 2023.

[9] C. Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of STOC*, pages 169–178, 2009.

[10] A. C. Yao. Protocols for secure computations. In *Proceedings of FOCS*, pages 160–164, 1982.

[11] J. Sun et al. zkLLM: Zero knowledge proofs for large language models. *arXiv:2404.16109*, 2024.

[12] Lagrange Labs. DeepProve-1: First full LLM zkML proof. 2025.

[13] PyO3 Project. PyO3: Rust bindings for the Python interpreter. https://pyo3.rs/, 2024.

[14] Apple Inc. Mobile Device Management Protocol Reference. *Apple Developer Documentation*, 2024.

[15] Apple Inc. Secure Enclave. *Apple Platform Security Guide*, 2024.

[16] Apple Inc. Managed Device Attestation. *WWDC22 Session 10143*, 2022.

[17] D. J. Bernstein, T. Lange, and P. Schwabe. The security impact of a new cryptographic library. In *LATINCRYPT*, pages 159–176, 2012.

[18] Smallstep. step-ca: An online certificate authority and ACME server. https://smallstep.com/docs/step-ca/, 2024.

[19] MicroMDM Project. MicroMDM: A macOS MDM server. https://micromdm.io/, 2024.

[20] Apple ML Research. MLX: An array framework for Apple Silicon. https://github.com/ml-explore/mlx, 2024.

[21] DGInf: Decentralized Private Inference on Apple Silicon. Open-source implementation. https://github.com/Layr-Labs/d-inference, 2026.